

# KYAMATU PRIMARY SCHOOL MANAGEMENT SYSTEM (KYAMATU-SMS)

## TEST PLAN AND TEST PROCEDURE SPECIFICATION

| Field            | Detail                                                 |
|------------------|--------------------------------------------------------|
| Student Name     | Musa Muthami Mwange                                    |
| Registration No. | 23/05037                                               |
| Course           | STU 4201 — Final Year Project II                       |
| Supervisor       | Deborah Mbula                                          |
| Institution      | KCA University                                         |
| Trimester        | Trimester 3, 2026                                      |
| Stream           | A (PTDL)                                               |
| Project Title    | Kyamatu Primary School Management System (KYAMATU-SMS) |
| Document Version | 2.0 (Standard IEEE / Academic Format)                  |
| Date             | October 2026                                           |

## TABLE OF CONTENTS

- [1.0 Introduction](#10-introduction)
- [1.1 Goals and Objectives](#11-goals-and-objectives)
- [1.2 Statement of Scope](#12-statement-of-scope)
- [1.3 Major Constraints](#13-major-constraints)
- [2.0 Test Plan](#20-test-plan)
- [2.1 Software (SCIs) to be Tested](#21-software-scis-to-be-tested)
- [2.2 Testing Strategy](#22-testing-strategy)

- [2.2.1 Unit Testing](#221-unit-testing)
- [2.2.2 Integration Testing](#222-integration-testing)
- [2.2.3 Validation Testing](#223-validation-testing)
- [2.2.4 High-Order Testing](#224-high-order-testing)
- [2.3 Testing Resources and Staffing](#23-testing-resources-and-staffing)
- [2.4 Test Work Products](#24-test-work-products)
- [2.5 Test Record Keeping](#25-test-record-keeping)
- [2.6 Test Metrics](#26-test-metrics)
- [2.7 Testing Tools and Environment](#27-testing-tools-and-environment)
- [2.8 Test Schedule](#28-test-schedule)
- [3.0 Test Procedure](#30-test-procedure)
- [3.1 Software (SCIs) to be Tested](#31-software-scis-to-be-tested)
- [3.2 Testing Procedure](#32-testing-procedure)
  - [3.2.1 Unit Test Cases](#321-unit-test-cases)
    - [3.2.1.1 Component 1: Authentication & Token Service](#3211-component-1-authentication--token-service)
    - [3.2.1.2 Component 2: Student Management & Admission Service](#3212-component-2-student-management--admission-service)
    - [3.2.1.3 Component 3: CBC Assessment & Grading Service](#3213-component-3-cbc-assessment--grading-service)
    - [3.2.1.4 Component 4: Fee Ledger & Invoicing Service](#3214-component-4-fee-ledger--invoicing-service)
    - [3.2.1.5 Component 5: Daily Attendance Service](#3215-component-5-daily-attendance-service)
  - [3.2.2 Integration Testing](#322-integration-testing)
    - [3.2.2.1 Testing Procedure for Integration](#3221-testing-procedure-for-integration)
    - [3.2.2.2 Stubs and Drivers Required](#3222-stubs-and-drivers-required)
    - [3.2.2.3 Integration Test Cases and Their Purpose](#3223-integration-test-cases-and-their-purpose)
    - [3.2.2.4 Expected Results](#3224-expected-results)
  - [3.2.3 Validation Testing](#323-validation-testing)

- [3.2.3.1 Testing Procedure for Validation](#3231-testing-procedure-for-validation)
- [3.2.3.2 Validation Test Cases (SRS Requirement Traceability)](#3232-validation-test-cases-srs-requirement-traceability)
- [3.2.3.3 Expected Results](#3233-expected-results)
- [3.2.3.4 Pass/Fail Criterion for All Validation Tests](#3234-passfail-criterion-for-all-validation-tests)
- [3.2.4 High-Order Testing (System Testing)](#324-high-order-testing-system-testing)
- [3.2.4.1 Recovery Testing](#3241-recovery-testing)
- [3.2.4.2 Security Testing](#3242-security-testing)
- [3.2.4.3 Stress Testing](#3243-stress-testing)
- [3.2.4.4 Performance Testing](#3244-performance-testing)
- [3.2.4.5 Alpha / Beta Testing](#3245-alpha--beta-testing)
- [3.2.4.6 Pass/Fail Criterion for High-Order Tests](#3246-passfail-criterion-for-high-order-tests)
- [3.3 Testing Resources and Staffing (ITG Definition)](#33-testing-resources-and-staffing-itg-definition)
- [3.4 Test Work Products](#34-test-work-products)
- [3.5 Test Record Keeping and Test Log](#35-test-record-keeping-and-test-log)

## 1.0 INTRODUCTION

This document provides a comprehensive overview of the testing lifecycle for the **Kyamatu Primary School Management System (KYAMATU-SMS)**. It unifies both the strategic **Test Plan** (governing testing philosophy, resource allocation, and milestone schedules) and the tactical **Test Procedure** (specifying stubs, test drivers, detailed test cases, execution steps, and pass/fail criteria).

### 1.1 Goals and Objectives

The primary goals and objectives of the software testing process are:

1. **Verification of Requirements:** Validate that every functional and non-functional requirement documented in the Software Requirements Specification (SRS) is completely and accurately realized.
2. **Defect Discovery and Elimination:** Uncover and rectify logical flaws, boundary condition bugs, data integrity inconsistencies, and race conditions prior to institutional cutover.
3. **CBC Pedagogical Alignment:** Confirm the absolute correctness of Competency-Based Curriculum (CBC) assessment ratings (Exceeding Expectations [EE], Meeting Expectations [ME], Approaching Expectations [AE], and Below Expectations [BE]), student ranking, and class mean calculations.
4. **Security and Access Governance:** Guarantee that Role-Based Access Control (RBAC) across six user roles (Super Admin, Admin, Teacher, Bursar, Student, Parent) strictly prohibits unauthorized privilege escalation and information leakage.
5. **System Robustness and Fault-Tolerance:** Establish high availability, graceful degradation under intermittent internet connectivity, and rapid crash recovery.

## 1.2 Statement of Scope

### 1.2.1 Features and Behaviors to be Tested

- **User Authentication & Session Management:** JWT issuance, 15-minute token expiry, 7-day refresh token rotation, password hashing with bcrypt, role resolution.
- **Student Information & Admissions:** New learner intake, Unique Learner Identifier (NEMIS UPI) mapping, birth certificate validation, multi-guardian linking, and class promotion workflows.
- **Academic & Curriculum Setup:** CBC grade tiers (PP1 through Grade 9), stream grouping, subject-to-grade assignment, and weekly timetable scheduling.
- **Daily Attendance Tracking:** Register marking (Present, Absent, Late, Excused), duplicate prevention, and class-wide attendance statistical summaries.

- **CBC Assessments & Automated Grading:** Continuous Assessment Tests (CATs), terminal exams, multi-dimensional rubric scoring, weighted aggregation, and rank generation.
- **Terminal Report Card Generation:** PDF document compilation, teacher and headteacher remarks, and printable layout verification.
- **Financial Accounting & Fee Invoicing:** Termly fee structure generation, automated student invoice compilation, cash/bank payment logging, and Safaricom M-Pesa Daraja STK Push callbacks.
- **Communication & Student Assistant:** Role-targeted school circulars, user-to-user messaging, and Gemini Pro AI chat integration.

### 1.2.2 Features and Behaviors NOT to be Tested (Exclusions)

- **External Payment Gateway Core Infrastructure:** The internal banking systems of Safaricom M-Pesa or commercial banks are excluded (mock stubs and sandbox APIs are utilized).
- **Physical Native Mobile Apps:** The future Phase 6 Flutter parent mobile application is outside the scope of this web release.
- **Hardware-Level Sensor Diagnostics:** Physical testing of server motherboard circuits or network switches beyond standard TCP/IP ping and load checks.

## 1.3 Major Constraints

1. **Academic Calendar Constraints:** Testing must adhere to the KCA University trimester calendar and coordinate with Kyamatu Primary School term operating dates.
2. **Rural Connectivity Limitations:** Kyamatu Primary School operates in an environment with occasional power fluctuations and bandwidth limitations; testing must validate low-bandwidth tolerance and local network resilience.
3. **Sandbox Rate Limits:** Safaricom Daraja API and Google Gemini API enforce development sandbox transaction rate quotas that constrain automated stress testing ceilings.
4. **Data Privacy Regulations:** In accordance with the Kenya Data Protection Act (2019), testing must use 100% fictionalized synthetic student data to protect learner identities.

## 2.0 TEST PLAN

### 2.1 Software (SCIs) to be Tested

The Software Configuration Items (SCIs) subject to testing encompass:

- **SCI-01 (Auth Module):** backend/src/features/auth/\* (Login, Token, Password Reset)
- **SCI-02 (Student Module):** backend/src/features/students/\* (Admissions, Profiles, Promotion)
- **SCI-03 (Staff Module):** backend/src/features/staff/\* (Workloads, Subject Assignments)
- **SCI-04 (Academic Module):** backend/src/features/academic/\* (Grades, Classes, Timetable, Lessons)
- **SCI-05 (Attendance Module):** backend/src/features/attendance/\* (Daily Register, Class Reports)
- **SCI-06 (Assessment Module):** backend/src/features/assessments/\* (Rubrics, CATs, Terminal Scoring)
- **SCI-07 (Reports Module):** backend/src/features/reports/\* (Rankings, PDFKit Compiler)
- **SCI-08 (Finance Module):** backend/src/features/fees/\* (Fee Structures, Invoices, M-Pesa Webhook)
- **SCI-09 (Communication Module):** backend/src/features/communication/\* (Announcements, Messages)
- **SCI-10 (Database Tier):** backend/prisma/schema.prisma (PostgreSQL 24 Tables, Enums, Indexes)
- **SCI-11 (Frontend SPA):** frontend/src/\* (React 18 components, Zustand state, Axios clients)

*Exclusions:* Third-party library internals (React, Express, Prisma engine binaries) are assumed reliable and are not subject to source-code testing.

## 2.2 Testing Strategy

### 2.2.1 Unit Testing Strategy

Unit testing verifies the operational correctness of individual JavaScript functions, service classes, and data transformers in complete isolation. All business logic in service files (\*.service.js) and validation rules (\*.validator.js) are targeted. Components with complex algorithms (such as the CBC grade conversion engine and the fee balance aggregator) are prioritized for 100% decision and branch coverage.

### 2.2.2 Integration Testing Strategy

Integration testing evaluates the data flow and interface communication across adjacent modules. A **Top-Down and Sandwich Integration approach** is employed:

1. **Core Backbone Integration:** Authentication middleware is integrated with the Express routing pipeline.
2. **Academic Pipeline Integration:** Classes & Subjects  $\rightarrow$  Timetable  $\rightarrow$  Attendance  $\rightarrow$  Assessments  $\rightarrow$  Report Cards.
3. **Financial Pipeline Integration:** Fee Structure  $\rightarrow$  Student Enrollment  $\rightarrow$  Term Invoices  $\rightarrow$  Payment Processing.
4. **Database Cascade Integration:** Verifying that record updates and cascading deletions maintain strict referential integrity across related Prisma models.

### 2.2.3 Validation Testing Strategy

Validation testing evaluates software compliance against functional business expectations outlined in the SRS. Validation follows a Black-Box approach, tracing user requirements directly to acceptance criteria. It validates that end-to-end user workflows (e.g., admitting a pupil, entering terminal scores, computing mean scores, generating report cards) behave as expected by school personnel.

### 2.2.4 High-Order Testing Strategy

High-order testing evaluates the fully integrated software system within its operational environment:

- **Recovery Testing:** Verifies the system's ability to recover data integrity following power outages, database process termination, and network disruptions.
- **Security Testing:** Verifies authentication mechanisms, attempts SQL injection and XSS exploits, and audits role boundaries.
- **Stress & Performance Testing:** Assesses API latency, query execution speed, and concurrency behavior under simulated peak load.
- **Alpha & Beta Testing:** Controlled trial runs with selected teachers and administrative staff from Kyamatu Primary School.

### 2.3 Testing Resources and Staffing

- **Test Lead / Developer:** Musa Muthami Mwange (Test scripting, unit execution, defect debugging).
- **Academic Supervisor / Quality Reviewer:** Deborah Mbula (Review of test procedures, verification of academic rigor).
- **Independent Testing Group (ITG):** Peer student testers and selected school staff conducting objective Black-Box validation.
- **Hardware Resources:** Dedicated test workstation (Intel Core i5, 16GB RAM) and test database instance running inside Docker.

### 2.4 Test Work Products

The testing process produces the following documented artifacts:

1. Master Test Plan and Test Procedure Specification (this document).
2. Automated Test Scripts (backend/\_\_tests\_\_/\*.test.js).
3. Chronological Test Execution Log (TEST\_LOG.md).
4. GitHub Issues Defect Tracking Registry.
5. User Acceptance Testing (UAT) Sign-Off Certificates.
6. Comprehensive Final Test Summary Report.

## 2.5 Test Record Keeping

All automated test runs are logged to standard output and stored in continuous integration log archives. Manual validation results are recorded in tabular test logs with execution timestamps, tester signatures, observed inputs, actual outputs, and categorical pass/fail determinations. Defects are logged immediately in the GitHub Issues tracking system with step-by-step reproduction instructions.

## 2.6 Test Metrics

The testing lifecycle is monitored using formal software engineering metrics:

- **Test Execution Rate:**  $\frac{\text{Executed Test Cases}}{\text{Total Planned Test Cases}} \times 100\%$  (Target: 100%).
- **Test Pass Rate:**  $\frac{\text{Passed Test Cases}}{\text{Executed Test Cases}} \times 100\%$  (Target:  $\geq 95\%$ ).
- **Defect Density:**  $\frac{\text{Confirmed Defects}}{\text{KLOC (Thousand Lines of Code)}}$  (Target:  $< 2.0 \text{ defects/KLOC}$ ).
- **Mean Time to Repair (MTTR):** Average time elapsed from bug identification to verified resolution (Target:  $< 24 \text{ hours}$  for High/Critical).

## 2.7 Testing Tools and Environment

- **Unit & Integration Framework:** Jest v29 + Supertest v7.
- **API Manual Validation:** Postman API Desktop Client + Swagger OpenAPI Documentation.
- **Browser Automation:** Cypress v13 for end-to-end UI user flows.
- **Test Environment Host:** Local Linux container (matundu\_test PostgreSQL 15 database) running on port 5432, completely isolated from production.

## 2.8 Test Schedule

| Testing Phase | Start Date  | End Date    | Primary Responsibility | Deliverable           |
|---------------|-------------|-------------|------------------------|-----------------------|
| Unit Testing  | 06 Oct 2026 | 16 Oct 2026 | Musa Muthami Mwange    | Unit Test Suite & Log |

|                                  |             |             |                      |                               |
|----------------------------------|-------------|-------------|----------------------|-------------------------------|
| <b>Integration Testing</b>       | 17 Oct 2026 | 24 Oct 2026 | Musa Muthami Mwange  | Integration Assessment Report |
| <b>Validation Testing</b>        | 25 Oct 2026 | 02 Nov 2026 | ITG & Developer      | SRS Traceability Verification |
| <b>High-Order System Tests</b>   | 03 Nov 2026 | 10 Nov 2026 | Musa Muthami Mwange  | Security & Stress Benchmarks  |
| <b>Alpha / Beta Field Trials</b> | 11 Nov 2026 | 18 Nov 2026 | School Staff & Admin | UAT Sign-Off Form             |
| <b>Final Documentation</b>       | 19 Nov 2026 | 22 Nov 2026 | Musa Muthami Mwange  | Final Test Summary Report     |

### 3.0 TEST PROCEDURE

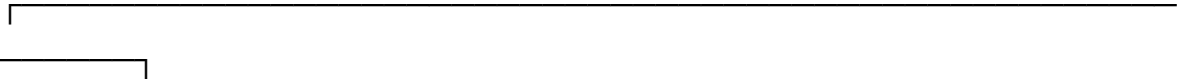
#### 3.1 Software (SCIs) to be Tested

This section outlines the detailed tactical test procedures for all software configuration items identified in Section 2.1. Exclusions remain external vendor banking networks and non-implemented Phase 6 native mobile apps.

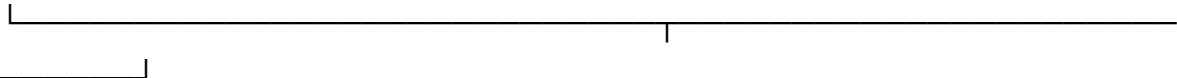
#### 3.2 Testing Procedure

|                                                     |  |
|-----------------------------------------------------|--|
|                                                     |  |
|                                                     |  |
| 3.2.1 Unit Testing (Isolating Functions & Services) |  |
|                                                     |  |

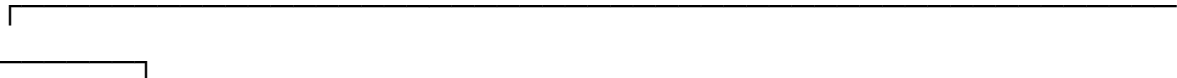
| Driver / Stub Interfaces



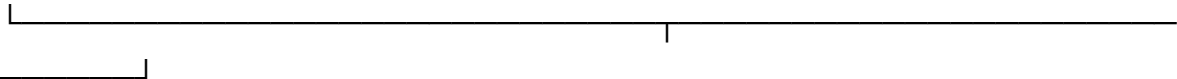
| 3.2.2 Integration Testing (Interface & Cascade Flow) |



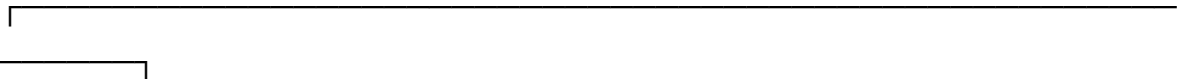
| Middleware & DB Verification



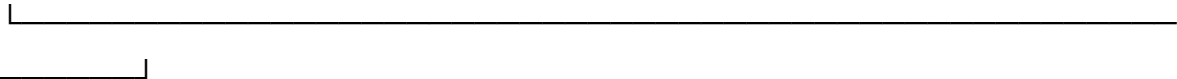
| 3.2.3 Validation Testing (SRS Business Requirements) |



| End-to-End User Simulation



| 3.2.4 High-Order Testing (Security, Stress, Recovery) |



### 3.2.1 Unit Test Cases

#### 3.2.1.1 Component 1: Authentication & Token Service (`auth.service.js`)

- **Stubs & Drivers:** Driver: Jest test suite invoking `authService.login()` and `authService.refreshToken()`. Stub: In-memory Prisma User mock returning pre-hashed bcrypt strings.
- **Purpose of Tests:** Verify that credential checking, password matching, and JWT creation operate securely and reject unauthorized requests.

| TC-ID   | Test Case Name                  | Test Inputs                                                      | Expected Results                                                |
|---------|---------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------|
| UTC-01A | Valid User Login                | email: "admin@kyamatu.a<br>c.ke", password: "Admin@123"          | Returns<br>accessToken (15m)<br>and refreshToken<br>(7d)        |
| UTC-01B | Invalid Password<br>Rejection   | email: "admin@kyamatu.a<br>c.ke", password: "WrongPass"          | Throws 401<br>Unauthorized; zero<br>tokens issued               |
| UTC-01C | Non-Existent<br>Account Login   | email: "ghost@kyamatu.a<br>c.ke", password: "Admin@123"          | Throws 401<br>Unauthorized;<br>uniform generic<br>error message |
| UTC-01D | Refresh Token<br>Regeneration   | Valid, unexpired<br>refresh token UUID<br>string                 | Returns newly<br>minted access token;<br>rotates refresh token  |
| UTC-01E | Expired Refresh<br>Token Reject | Token timestamp<br>expired ( $\$ > \backslash$<br>text{ days}\$) | Throws 401 Token<br>Expired; session<br>terminated              |

### 3.2.1.2 Component 2: Student Management & Admission Service (`students.service.js`)

- **Stubs & Drivers:** Driver: Jest test harness executing `createStudent()` and `approveAdmission()`. Stub: Mock Prisma database client.
- **Purpose of Tests:** Validate student record integrity, ensure admission uniqueness, and verify guardian linking.

| TC-ID   | Test Case Name                 | Test Inputs                                                                | Expected Results                                               |
|---------|--------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------|
| UTC-02A | Create Valid Student           | Full payload,<br>admissionNumber:<br>"KYA/2026/042"                        | Student record<br>instantiated; status<br>set to PENDING       |
| UTC-02B | Duplicate Admission<br>Number  | Existing<br>admissionNumber:<br>"KYA/2026/042"                             | Throws 409<br>Conflict; unique<br>constraint preserved         |
| UTC-02C | Admission Status<br>Transition | studentId:<br>valid_uuid, action:<br>APPROVE                               | Status updates from<br>PENDING to<br>APPROVED                  |
| UTC-02D | Link Primary<br>Guardian       | studentId:<br>valid_uuid,<br>guardianId:<br>valid_uuid                     | Junction table<br>StudentGuardian<br>created with<br>isPrimary |
| UTC-02E | Age Boundary<br>Enforcement    | Date of birth set to<br>future date ( $\$t > \backslash$<br>text{today}\$) | Throws 400<br>Validation Error;<br>rejection enforced          |

### 3.2.1.3 Component 3: CBC Assessment & Grading Service (`assessments.service.js`)

- **Stubs & Drivers:** Driver: Test script executing score calculation functions. Stub: Hardcoded subject rubrics.
- **Purpose of Tests:** Ensure numerical mark inputs strictly evaluate to accurate CBC ratings and traditional letter grades.

| TC-ID | Test Case Name | Test Inputs | Expected Results |
|-------|----------------|-------------|------------------|
|-------|----------------|-------------|------------------|

|                |                           |                                            |                                              |
|----------------|---------------------------|--------------------------------------------|----------------------------------------------|
| <b>UTC-03A</b> | CBC EE Rating Boundary    | Score: \$82\%\$ on Grade 5 Mathematics CAT | Assigned Rating: EXCEEDING (EE); Grade: A    |
| <b>UTC-03B</b> | CBC ME Rating Boundary    | Score: \$64\%\$ on Grade 4 English CAT     | Assigned Rating: MEETING (ME); Grade: C      |
| <b>UTC-03C</b> | CBC AE Rating Boundary    | Score: \$38\%\$ on Grade 6 Science CAT     | Assigned Rating: APPROACHING (AE); Grade: E  |
| <b>UTC-03D</b> | CBC BE Rating Boundary    | Score: \$18\%\$ on Grade 3 Kiswahili CAT   | Assigned Rating: BELOW (BE); Grade: E        |
| <b>UTC-03E</b> | Out-of-Bounds Score Entry | Score: \$115\%\$ (Max Score: \$100\$)      | Throws 400 Validation Error; input discarded |

#### 3.2.1.4 Component 4: Fee Ledger & Invoicing Service (`fees.service.js`)

- **Stubs & Drivers:** Driver: Unit test runner invoking `generateInvoice()` and `recordPayment()`. Stub: Synthetic student financial balance ledger.
- **Purpose of Tests:** Confirm that invoice totals, partial payments, and balances calculate accurately.

| TC-ID          | Test Case Name             | Test Inputs                                          | Expected Results                                                                        |
|----------------|----------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>UTC-04A</b> | Term Invoice Generation    | Grade 5 fee structure: Tuition 8,000, Activity 2,000 | Generates invoice with <code>totalAmount</code> : 10,000, <code>balance</code> : 10,000 |
| <b>UTC-04B</b> | Partial Cash Payment Entry | Invoice Balance: 10,000; Payment: 4,000 (CASH)       | <code>paidAmount</code> : 4,000, new <code>balance</code> : 6,000                       |

|                |                             |                                                |                                                        |
|----------------|-----------------------------|------------------------------------------------|--------------------------------------------------------|
|                |                             |                                                | recorded                                               |
| <b>UTC-04C</b> | Full Fee Settlement         | Invoice Balance: 6,000; Payment: 6,000 (MPESA) | paidAmount: 10,000, new balance: 0.00 recorded         |
| <b>UTC-04D</b> | Duplicate Receipt Reference | M-Pesa receipt QHJ89912K entered twice         | Throws 409 Conflict on transaction reference duplicate |

### 3.2.1.5 Component 5: Daily Attendance Service (`attendance.service.js`)

- **Stubs & Drivers:** Driver: Test script executing `markAttendance()`. Stub: Synthetic class roster.
- **Purpose of Tests:** Validate attendance marking, daily record locking, and percentage aggregations.

| TC-ID          | Test Case Name             | Test Inputs                                              | Expected Results                                            |
|----------------|----------------------------|----------------------------------------------------------|-------------------------------------------------------------|
| <b>UTC-05A</b> | Daily Register Submission  | 40 students marked (38 PRESENT, 2 ABSENT) for Date \$D\$ | 40 attendance rows created in database                      |
| <b>UTC-05B</b> | Duplicate Same-Day Marking | Attempt to submit attendance again for student on \$D\$  | Rejected by database unique composite key (studentId, date) |
| <b>UTC-05C</b> | Attendance Percentage Calc | Student with 76 days present out of 80 school days       | Evaluates precisely to 95.0%\%                              |

## 3.2.2 Integration Testing

### 3.2.2.1 Testing Procedure for Integration

Integration testing validates interface handoffs by firing HTTP requests into the Express middleware stack, traversing the service layer, and evaluating PostgreSQL database mutations. Tests run in order of foundational dependencies:

1. Auth Middleware  $\rightarrow$  All Protected Feature Routes.
2. Student Admission  $\rightarrow$  Class Enrollment  $\rightarrow$  Invoice Generation.
3. Assessment Scores  $\rightarrow$  Class Ranking Engine  $\rightarrow$  PDF Report Compiler.

### 3.2.2.2 Stubs and Drivers Required

- **Test Driver:** Supertest HTTP agent injecting requests into Express `app.listen()`.
- **Payment Gateway Stub:** A mock M-Pesa Daraja callback server simulating Safaricom's confirmation JSON payload.
- **AI Service Stub:** Mock Gemini Pro client returning deterministic responses during offline test execution.

### 3.2.2.3 Integration Test Cases and Their Purpose

| ITC-ID | Integration Interface Flow                    | Purpose of Test                                        | Expected Results                                        |
|--------|-----------------------------------------------|--------------------------------------------------------|---------------------------------------------------------|
| ITC-01 | RBAC Middleware $\rightarrow$ Fee Endpoints   | Ensure Teacher/Student roles cannot invoke fee billing | HTTP 403 Forbidden; zero changes to database            |
| ITC-02 | Student Enrollment $\rightarrow$ Term Billing | Verify invoice auto-creation upon student admission    | StudentInvoice record auto-generated matching grade fee |
| ITC-03 | Score Entry $\rightarrow$                     | Confirm aggregated                                     | Report card displays                                    |

|               |                                                    |                                                    |                                                           |
|---------------|----------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------|
|               | rightarrow\$ Report Card Summary                   | marks match individual CAT entries                 | exact weighted sum of student marks                       |
| <b>ITC-04</b> | M-Pesa Webhook \$\\rightarrow\$ Ledger Balance     | Verify payment callback increments invoice ledger  | Payment status updates to COMPLETED; balance decrements   |
| <b>ITC-05</b> | Student Deletion \$\\rightarrow\$ Database Cascade | Confirm referential integrity upon student removal | Cascading delete removes scores, invoices, and attendance |

#### 3.2.2.4 Expected Results

All cross-module transactions complete without orphaned foreign key references. Database transactions wrap multi-table writes: if any step fails, the entire transaction rolls back cleanly.

### 3.2.3 Validation Testing

#### 3.2.3.1 Testing Procedure for Validation

Validation testing validates that the system fulfills all functional requirements in the Software Requirements Specification (SRS). Test procedures simulate real operational scenarios at Kyamatu Primary School using an interactive browser and representative test datasets.

#### 3.2.3.2 Validation Test Cases (SRS Requirement Traceability)

| <b>VTC-ID</b> | <b>SRS Requirement Traced</b>             | <b>Operational Validation Scenario</b>            | <b>Expected Results</b>                          |
|---------------|-------------------------------------------|---------------------------------------------------|--------------------------------------------------|
| <b>VTC-01</b> | <b>REQ-01:</b> Secure User Authentication | Administrator and teacher login via web interface | User lands on designated role-tailored dashboard |
| <b>VTC-02</b> | <b>REQ-02:</b> Learner                    | Admission of new                                  | Profile visible in                               |

|               |                                                 |                                                          |                                                           |
|---------------|-------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------|
|               | Admission & Profiles                            | Grade 1 pupil with birth certificate                     | Grade 1 class register roster                             |
| <b>VTC-03</b> | <b>REQ-03:</b> Daily Attendance Marking         | Teacher marks daily register for Grade 5                 | Daily presence statistics update on admin dashboard       |
| <b>VTC-04</b> | <b>REQ-04:</b> CBC Assessment Recording         | Teacher records CAT 1 & CAT 2 marks for English          | EE/ME/AE/BE ratings automatically populated in gradebook  |
| <b>VTC-05</b> | <b>REQ-05:</b> Terminal Report Card Generation  | Class teacher triggers End-of-Term report cards          | Printable PDF generated with rankings, marks, and remarks |
| <b>VTC-06</b> | <b>REQ-06:</b> Term Fee Collection & Receipting | Bursar records cash fee payment of KES 5,000             | Printable receipt generated; outstanding balance updated  |
| <b>VTC-07</b> | <b>REQ-07:</b> School Broadcast Announcements   | Headteacher publishes circular to all parents            | Circular rendered on student and parent portal dashboards |
| <b>VTC-08</b> | <b>REQ-08:</b> Timetable Conflict Detection     | Admin assigns teacher to two different classes same hour | System blocks assignment with teacher conflict warning    |

### 3.2.3.3 Expected Results

Each business feature executes within 3 user clicks or fewer from its parent dashboard, and all user-facing calculations match manual audit checks.

#### 3.2.3.4 Pass/Fail Criterion for All Validation Tests

- **Pass Criteria:** 100% of tested SRS requirements execute with zero critical software crashes; mathematical calculations (fees and mean grades) match manual calculations with zero deviation.
- **Fail Criteria:** Any unhandled 500 Internal Server Error, incorrect student ranking, or security bypass constitutes a test failure.

### 3.2.4 High-Order Testing (System Testing)

#### 3.2.4.1 Recovery Testing

- **Test Procedure:** Terminate the PostgreSQL database service process abruptly during an active bulk score entry transaction.
- **Specialized Requirements:** Simulated power-cut scenario via process kill commands.
- **Pass/Fail Criteria:** PASS if the database recovers within 30 seconds upon restart with zero corrupted partial records, and the user receives a clean error message rather than a hung connection.

#### 3.2.4.2 Security Testing

- **Test Procedure:** Execute automated vulnerability scans using OWASP ZAP and custom injection scripts:
- *SQL Injection:* Submitting ' OR 1=1 -- into login email fields and search inputs.
- *XSS Scripting:* Injecting <script>alert('XSS')</script> into announcement posts.
- *Privilege Escalation:* Student token attempting POST /api/fees/structures.
- **Pass/Fail Criteria:** PASS if all SQL injection attempts are neutralized by Prisma parameterized queries, XSS strings are sanitized, and unauthorized requests return HTTP 403 Forbidden.

#### 3.2.4.3 Stress Testing

- **Test Procedure:** Simulate continuous load beyond standard operating capacity using Apache Bench (ab):
- Inundate the login endpoint with 1,500 requests across 50 concurrent connections.

- **Pass/Fail Criteria:** PASS if the Redis-backed rate limiter throttles excessive requests with HTTP 429 Too Many Requests, preventing CPU saturation and server crashes.

#### 3.2.4.4 Performance Testing

- **Test Procedure:** Benchmark response latency across key endpoints under standard load (20 concurrent users):
- GET /api/students (paginated list of 500 pupils).
- POST /api/reports/generate (generating terminal reports for 45 students).
- **Pass/Fail Criteria:** PASS if API response times meet performance benchmarks:
- Simple queries:  $< 300\text{ ms}$ .
- Complex report card aggregation:  $< 2.0\text{ seconds}$ .
- Frontend Lighthouse performance rating:  $\geq 80/100$ .

#### 3.2.4.5 Alpha / Beta Testing

- **Alpha Testing:** Conducted in an isolated lab environment by peer developers and project supervisor Deborah Mbula to uncover defects and interface inconsistencies.
- **Beta Testing:** Conducted on-site at Kyamatu Primary School with the Headteacher, 2 class teachers, and the bursar operating the software on real school laptops with test datasets.
- **Pass/Fail Criteria:** PASS if beta testers achieve a System Usability Scale (SUS) satisfaction score of  $\geq 75/100$  and report no blocking usability hurdles.

#### 3.2.4.6 Pass/Fail Criterion for High-Order Tests

The system passes High-Order System Testing if and only if:

1. Zero high or critical security vulnerabilities exist.
2. The system handles simulated traffic spikes without unhandled process crashes.
3. System recovery from abrupt termination completes with verified ACID data integrity.

### 3.3 Testing Resources and Staffing (ITG Definition)

#### 3.3.1 Resource Allocation Matrix

- **Testing Lead (Developer):** Musa Muthami Mwange — test environment management, test automation execution, debugging.
- **Supervisor & Quality Auditor:** Deborah Mbula — academic validation, testing milestone reviews.
- **Independent Testing Group (ITG):** An external testing cohort consisting of two KCA University computing student peers and designated staff from Kyamatu Primary School (Headteacher and Lead Teacher).

#### 3.3.2 Role and Mandate of the ITG

The ITG operates independently from daily development activities. Their responsibilities include:

1. Conducting unbiased black-box usability and validation testing.
2. Executing adversarial testing designed to uncover edge-case failures.
3. Confirming that report cards and fee receipt layouts conform to Ministry of Education and Kyamatu Primary standards.
4. Providing formal sign-off on the User Acceptance Testing (UAT) milestone.

### 3.4 Test Work Products

Upon completion of all test procedures, the following deliverables are archived:

- `4_Test_Plan.pdf` / `Deborah_Mbula_Test_Plan.pdf` — Approved master test documentation.
- Automated Test Execution Reports — Jest test runner summary outputs with percentage coverage metrics.
- Security Audit Checklist — Results of OWASP vulnerability scans and penetration tests.

- Chronological Defect Log — Complete defect register detailing bug descriptions, severity tiers, and resolution commits.
- Formal UAT Sign-Off Certificate — Signed by Kyamatu Primary School administration.

### 3.5 Test Record Keeping and Test Log

All tests conducted are entered chronologically into the Master System Test Log. The log serves as an auditable verification trail demonstrating testing rigor:

| Log ID  | Date & Time          | Component / Test Suite  | Executed By     | Environment    | Pass / Fail | Defects Logged |
|---------|----------------------|-------------------------|-----------------|----------------|-------------|----------------|
| LOG-101 | 06 Oct 2026<br>10:00 | Unit: Auth Service      | Musa Muthami M. | Docker Test DB | PASS        | None           |
| LOG-102 | 07 Oct 2026<br>14:30 | Unit: Students Service  | Musa Muthami M. | Docker Test DB | PASS        | None           |
| LOG-103 | 09 Oct 2026<br>11:15 | Unit: Assessment Engine | Musa Muthami M. | Docker Test DB | PASS        | None           |
| LOG-104 | 12 Oct 2026<br>16:00 | Unit: Fee Ledger        | Musa Muthami M. | Docker Test DB | PASS        | None           |
| LOG-105 | 18 Oct 2026<br>09:45 | Integration : RBAC Flow | Musa Muthami M. | Local Node/PG  | PASS        | None           |
| LOG-106 | 21 Oct 2026<br>15:20 | Integration : Reports   | Musa Muthami M. | Local Node/PG  | PASS        | None           |
| LOG-107 | 26 Oct               | Validation              | ITG Peer        | Staging        | PASS        | None           |

|                |                         |                                |                       |                 |             |      |
|----------------|-------------------------|--------------------------------|-----------------------|-----------------|-------------|------|
|                | 2026<br>11:00           | :<br>Admission<br>s            | Review                | Web             |             |      |
| <b>LOG-108</b> | 29 Oct<br>2026<br>14:00 | Validation<br>:<br>Gradebook   | ITG / Lead<br>Teacher | Staging<br>Web  | <b>PASS</b> | None |
| <b>LOG-109</b> | 04 Nov<br>2026<br>10:30 | High-<br>Order:<br>Security    | Musa<br>Muthami<br>M. | OWASP<br>ZAP    | <b>PASS</b> | None |
| <b>LOG-110</b> | 08 Nov<br>2026<br>16:00 | High-<br>Order:<br>Stress Test | Musa<br>Muthami<br>M. | Apache<br>Bench | <b>PASS</b> | None |
| <b>LOG-111</b> | 14 Nov<br>2026<br>14:00 | High-<br>Order:<br>Beta Field  | Kyamatu<br>Admin      | School<br>PCs   | <b>PASS</b> | None |

**Document Approved By:**

*Student Investigator:* Musa Muthami Mwange (23/05037) — Signature:

\_\_\_\_\_ Date: \_\_\_\_\_

*Project Supervisor:* Deborah Mbula — Signature: \_\_\_\_\_ Date:

\_\_\_\_\_